

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ
ΥΠΟΛΟΓΙΣΤΩΝ

ΣΧΕΔΙΑΣΜΟΣ ΚΑΙ ΥΛΟΠΟΙΗΣΗ
ΠΛΑΤΦΟΡΜΑΣ ΔΙΑΧΕΙΡΙΣΗΣ ΤΟΥΡΙΣΤΙΚΩΝ
ΚΑΤΑΛΥΜΑΤΩΝ ΜΕ ΕΜΦΑΣΗ ΣΤΟ RAPID
DEVELOPMENT ΚΑΙ ΤΗΝ ΕΝΣΩΜΑΤΩΣΗ
LLMs

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ

ΘΕΟΔΩΡΟΣ ΜΠΑΡΚΑΣ

ΧΑΝΙΑ, ΝΟΕΜΒΡΙΟΣ 2025

ΠΝΕΥΜΑΤΙΚΑ ΔΙΚΑΙΩΜΑΤΑ

«Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για μη κερδοσκοπικό σκοπό, εκπαιδευτικού ή ερευνητικού χαρακτήρα, με την προϋπόθεση να αναφέρεται η πηγή προέλευσης. Ερωτήματα που αφορούν τη χρήση της εργασίας για άλλη χρήση θα πρέπει να απευθύνονται προς τον συγγραφέα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και δεν πρέπει να ερμηνευθεί ότι αντιπροσωπεύουν τις επίσημες θέσεις του Πολυτεχνείου Κρήτης.»

ΕΞΕΤΑΣΤΙΚΗ ΕΠΙΤΡΟΠΗ

1. **Καθηγητής Μιχαήλ Γ. Λαγουδάκης** (Επιβλέπων)
Πολυτεχνείο Κρήτης, Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών
Υπολογιστών
2. **Καθηγητής Αντώνιος Δεληγιαννάκης**
Πολυτεχνείο Κρήτης, Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών
Υπολογιστών
3. **Αναπληρωτής Καθηγητής Βασίλειος Σαμολαδάς**
Πολυτεχνείο Κρήτης, Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών
Υπολογιστών

Περίληψη

Η παρούσα διπλωματική εργασία πραγματεύεται τον σχεδιασμό και την υλοποίηση της πλατφόρμας «Lunarety V2», ενός προηγμένου συστήματος διαχείρισης τουριστικών καταλυμάτων (Property Management System – PMS) που αξιοποιεί σύγχρονες τεχνολογίες για να προσφέρει λύσεις Rapid Development και ενσωμάτωση Τεχνητής Νοημοσύνης (LLMs). Σκοπός της εργασίας είναι η κάλυψη του κενού που υπάρχει στην αγορά για ένα εργαλείο που απευθύνεται σε διαχειριστές ακινήτων (Managers) οι οποίοι επιβλέπουν πολλαπλούς ιδιοκτήτες (Owners), προσφέροντας διαφάνεια, αυτοματισμό και ευκολία χρήσης που λείπει από τα παραδοσιακά Channel Managers.

Η μεθοδολογία ανάπτυξης βασίστηκε στη χρήση του PayloadCMS v3.0, το οποίο λειτουργεί ως ένα πλήρες backend framework εντός του Next.js, προσφέροντας Code-First ορισμό δομών δεδομένων και ισχυρή τυποποίηση μέσω TypeScript. Η βάση δεδομένων υλοποιήθηκε σε PostgreSQL με χρήση Drizzle ORM, εξασφαλίζοντας ακεραιότητα και ταχύτητα. Η πλατφόρμα ακολουθεί αρχιτεκτονική Monorepo και είναι σχεδιασμένη για Serverless Deployment (π.χ. Vercel), επιτρέποντας στους διαχειριστές να ξεκινούν με μηδενικό κόστος υποδομής και να κλιμακώνονται ανάλογα με τις ανάγκες τους.

Κεντρικό στοιχείο της καινοτομίας αποτελεί η ενσωμάτωση Large Language Models (LLMs) μέσω του Vercel AI SDK και του OpenRouter, προσφέροντας έξυπνες λειτουργίες σε πολλαπλά επίπεδα. Επιπλέον, αναπτύχθηκε ένας μηχανισμός «Auto Actions» για την πλήρη αυτοματοποίηση της επικοινωνίας μέσω πολλαπλών καναλιών (Email, OTA Messages, Telegram).

Η εργασία συνεισφέρει στην κατανόηση του τρόπου με τον οποίο οι σύγχρονες αρχιτεκτονικές λογισμικού και τα εργαλεία Rapid Development μπορούν να μετασχηματίσουν επιχειρησιακές διαδικασίες, προσφέροντας λύσεις υψηλής αξίας με μειωμένο χρόνο ανάπτυξης και συντήρησης.

Λέξεις-κλειδιά: Property Management System, Rapid Development, Large Language Models, PayloadCMS, Next.js, TypeScript, Serverless, Channel Manager, Auto Actions

Abstract

This diploma thesis deals with the design and implementation of the «Lunarety V2» platform, an advanced Property Management System (PMS) that leverages modern technologies to offer Rapid Development solutions and Large Language Model (LLM) integration. The purpose of this work is to bridge the market gap for a tool tailored to Property Managers overseeing multiple Owners, offering transparency, automation, and ease of use often missing from traditional Channel Managers.

The development methodology was based on PayloadCMS v3.0, acting as a full backend framework within Next.js, offering Code-First data structure definitions and strong typing via TypeScript. The database was implemented in PostgreSQL using Drizzle ORM, ensuring data integrity and performance. The platform follows a Monorepo architecture and is designed for Serverless Deployment (e.g., Vercel), allowing managers to start with zero infrastructure costs and scale as needed.

A key innovation element is the integration of Large Language Models (LLMs) via Vercel AI SDK and OpenRouter, offering smart features at multiple levels. Furthermore, an «Auto Actions» mechanism was developed to fully automate communication through multiple channels (Email, OTA Messages, Telegram).

This work contributes to understanding how modern software architectures and Rapid Development tools can transform business processes, delivering high-value solutions with reduced development and maintenance time.

Keywords: Property Management System, Rapid Development, Large Language Models, PayloadCMS, Next.js, TypeScript, Serverless, Channel Manager, Auto Actions

Πρόλογος

Θα ήθελα να ευχαριστήσω... (Placeholder for Acknowledgements)

Περιεχόμενα

Περίληψη	5
Abstract	7
Πρόλογος	9
Κατάλογος Εικόνων	15
Κατάλογος Πινάκων	17
1 Εισαγωγή	19
1.1 Αντικείμενο της Μελέτης	19
1.2 Σκοπός της Διπλωματικής Εργασίας	19
1.3 Το Κενό στην Αγορά: Managers και Owners	20
1.4 Συνεισφορά στο Rapid Development	20
2 Θεωρητικό Υπόβαθρο	21
2.1 Σύγχρονες Αρχιτεκτονικές Web	21
2.1.1 TypeScript και Type Consistency	22
2.1.2 PayloadCMS: Backend Framework vs CMS	22
2.1.3 Next.js, Server Actions και Monorepo	22
2.1.4 PostgreSQL και Drizzle ORM	23
2.1.5 Shadcn UI, React και Zustand	23
2.1.6 Swagger/OpenAPI και Code Generation	23
2.2 Large Language Models (LLMs)	23
2.2.1 Vercel AI SDK και Streaming	24
2.2.2 OpenRouter και Πρόσβαση σε Μοντέλα	24
2.3 Channel Managers και Διασυνδεσιμότητα	24
3 Μεθοδολογία και Ανάλυση Απαιτήσεων	25
3.1 Ρόλοι και Ιεραρχία Χρηστών	25
3.1.1 Τύποι Χρηστών (User Types)	25
3.1.2 Ιεραρχικές Σχέσεις και Billing	26
3.2 Αρχιτεκτονική Συστήματος (Monorepo)	27
3.2.1 Δομή Monorepo	27
3.2.2 Βασικά Components	28

3.2.3	External Integrations	29
3.3	Μοντέλο Δεδομένων και Σχέσεις	30
3.3.1	Κύριες Οντότητες (Collections)	30
3.3.2	Σχέσεις Μεταξύ Οντοτήτων	30
3.3.3	Hook System και Lifecycle	31
4	Υλοποίηση	33
4.1	Διαχείριση Κρατήσεων και Συγχρονισμός	33
4.1.1	Αρχική Σύνδεση και Ρύθμιση Webhooks	33
4.1.2	Μέθοδοι Εισαγωγής Κρατήσεων	34
4.1.2.1	A) Mass Import (Property Syncing)	35
4.1.2.2	B) Webhook Updates (Channel Manager Triggers)	35
4.1.2.3	Γ) UI-Based Booking Creation	36
4.1.2.4	Δ) Website Booking Creation	37
4.2	Μηχανισμός Auto Actions	37
4.2.1	Κατηγορίες Auto Actions	38
4.2.2	Κανάλια Επικοινωνίας	38
4.2.3	Μηχανισμός Προτύπων (Template Engine)	39
4.2.4	Time-based Triggers (Cron Jobs)	40
4.2.5	Αρχιτεκτονική Cron Jobs	41
4.3	Σύστημα Χρέωσης και Τιμολόγησης	42
4.3.1	Αρχιτεκτονική Χρέωσης	42
4.3.2	Ανάλυση Χρεώσεων	42
4.3.3	Σύστημα Ελέγχου Πρόσβασης	42
4.4	Αυτοματοποιημένη Παραγωγή Ιστοσελίδων	43
4.4.1	Δομή και Λειτουργία Website	43
4.4.2	Website API Endpoints	44
4.5	Περιβάλλον Χρήστη (Platform UI)	44
4.5.1	Dashboard και Calendar	44
4.5.2	Σύστημα Μηνυμάτων	44
5	Αποτελέσματα	45
5.1	Παρουσίαση της Πλατφόρμας	45
5.2	Παρουσίαση του Generated Website	46
5.3	Τεχνικές Προκλήσεις και Λύσεις	47
5.3.1	Συγχρονισμός Δεδομένων με External APIs	47
5.3.2	Αποφυγή Infinite Loops	47
5.3.3	Απόδοση Μαζικών Εισαγωγών	47

6 Συζήτηση	49
6.1 Αξιολόγηση Τεχνολογικών Επιλογών	49
6.2 Μελλοντικές Βελτιώσεις	49
6.2.1 Βελτίωση UX στο Admin Panel	49
6.2.2 Επέκταση Channel Manager Support	49
6.2.3 Mobile Application	50
6.2.4 Advanced Analytics Dashboard	50
7 Συμπεράσματα	51
7.1 Επίτευξη Στόχων	51
7.2 Τεχνικά Συμπεράσματα	51
7.3 Βασικά Διδάγματα	52
Βιβλιογραφία	53
A' Δομή Webhook Payload	55
B' Παράδειγμα Auto Action Query	57

Κατάλογος σχημάτων

2.1	Το τεχνολογικό stack της πλατφόρμας (Next.js, PayloadCMS, PostgreSQL, κ.α.).	21
3.1	Ιεραρχική δομή ρόλων χρηστών στην πλατφόρμα.	27
3.2	Διάγραμμα αρχιτεκτονικής συστήματος (Monorepo).	29
3.3	Entity Relationship Diagram (ERD) του μοντέλου δεδομένων της πλατφόρμας.	32
4.1	Μέθοδοι εισαγωγής κρατήσεων στην πλατφόρμα.	34
4.2	Τριπλή δομή προμηθειών (Three-Tier Commission Structure).	36
4.3	Ροή δεδομένων συγχρονισμού κρατήσεων με το Beds24.	37
4.4	Διάγραμμα ροής εκτέλεσης των Auto Actions.	39
4.5	Σενάρια λειτουργίας Time-based Triggers για κράτηση τελευταίας στιγμής.	41
4.6	Decision tree για τον έλεγχο πρόσβασης.	43
5.1	Η σελίδα του Ημερολογίου (Calendar).	45
5.2	Η σελίδα των Μηνυμάτων με AI capabilities.	45
5.3	Διαδικασία παραγωγής Website από το περιβάλλον του διαχειριστή.	46
5.4	Πλοήγηση επισκέπτη στην παραγόμενη ιστοσελίδα.	46
5.5	Δυνατότητες AI και LLM για την εξυπηρέτηση επισκεπτών.	47

Κατάλογος πινάκων

3.1	Τύποι χρηστών της πλατφόρμας	25
3.2	External Services και Integrations	29
3.3	Κύριες Collections της βάσης δεδομένων	30
3.4	Hook Types και χρήση στην πλατφόρμα	31
4.1	Υποστηριζόμενοι Time-based Trigger Types	40
4.2	Scheduled Cron Jobs της πλατφόρμας	41
4.3	Τύποι χρεώσεων Billing	42
4.4	Website API Endpoints	44
5.1	Σύγκριση στρατηγικών εισαγωγής	47
6.1	Αξιολόγηση τεχνολογικών επιλογών	49
7.1	Κύρια επιτεύγματα της πλατφόρμας	51

Κεφάλαιο 1

Εισαγωγή

1.1 Αντικείμενο της Μελέτης

Το αντικείμενο της παρούσας διπλωματικής εργασίας είναι ο σχεδιασμός και η ανάπτυξη ενός ολοκληρωμένου πληροφοριακού συστήματος, του «Lunarety V2», για τη διαχείριση τουριστικών καταλυμάτων. Η πλατφόρμα δεν αποτελεί απλώς ένα ακόμη PMS, αλλά μια ολιστική λύση που στοχεύει στην κεντροποίηση των λειτουργιών που απαιτούνται από επαγγελματίες διαχειριστές (Managers) και τους πελάτες τους, τους ιδιοκτήτες ακινήτων (Owners). Πέρα από τις βασικές λειτουργίες διαχείρισης κρατήσεων και συγχρονισμού ημερολογίων, η πλατφόρμα ενσωματώνει προηγμένες δυνατότητες Τεχνητής Νοημοσύνης (LLMs) για την υποβοήθηση της επικοινωνίας και την παροχή έξυπνων υπηρεσιών.

1.2 Σκοπός της Διπλωματικής Εργασίας

Ο κύριος σκοπός της εργασίας είναι η δημιουργία ενός εργαλείου που μειώνει δραστικά τον διοικητικό φόρτο και εκσυγχρονίζει τις διαδικασίες διαχείρισης. Ειδικότερα, επιδιώκεται:

- Η δημιουργία ενός robust συστήματος διαχείρισης σχέσεων Manager-Owner, προσφέροντας διαφάνεια και αυτοματοποιημένη ενημέρωση.
- Η ενσωμάτωση δυνατοτήτων LLM (Large Language Models) τόσο για την υποβοήθηση των χρηστών στη σύνταξη μηνυμάτων όσο και για την παροχή smart features στους επισκέπτες μέσω των ιστοσελίδων των καταλυμάτων.
- Η υλοποίηση ενός ευέλικτου συστήματος «Auto Actions» τριών επιπέδων για την πλήρη αυτοματοποίηση των εργασιών.
- Η απρόσκοπτη διασύνδεση με τον Channel Manager «Beds24», με αρχιτεκτονική πρόβλεψη για μελλοντική προσθήκη άλλων παρόχων.

1.3 Το Κενό στην Αγορά: Managers και Owners

Η παρούσα εργασία απευθύνεται κυρίως σε επαγγελματίες Property Managers που διαχειρίζονται χαρτοφυλάκια ακινήτων που ανήκουν σε τρίτους (Owners). Στην τρέχουσα αγορά, παρατηρείται ένα σημαντικό κενό:

- Οι Managers συχνά αναγκάζονται να χρησιμοποιούν πολύπλοκα λογισμικά Channel Managers, τα οποία είναι δύσκολα για τους Owners ή το απλό προσωπικό.
- Η ενημέρωση των Owners για κρατήσεις, πληρότητες και οικονομικά στοιχεία γίνεται συχνά χειροκίνητα (τηλέφωνα, excel, emails), οδηγώντας σε λάθη και καθυστερήσεις.
- Η δημιουργία λογαριασμών για Owners μέσα στα υπάρχοντα Channel Managers είναι συχνά δαπανηρή ή πολύπλοκη διαδικασία.

Το Lunarety V2 έρχεται να καλύψει αυτό το κενό, προσφέροντας μια ενδιαμέση, φιλική προς τον χρήστη πλατφόρμα, όπου ο Manager ορίζει τα δικαιώματα, τις συνδρομές και την πρόσβαση των Owners, παρέχοντάς τους ακριβώς την πληροφορία που χρειάζονται σε πραγματικό χρόνο.

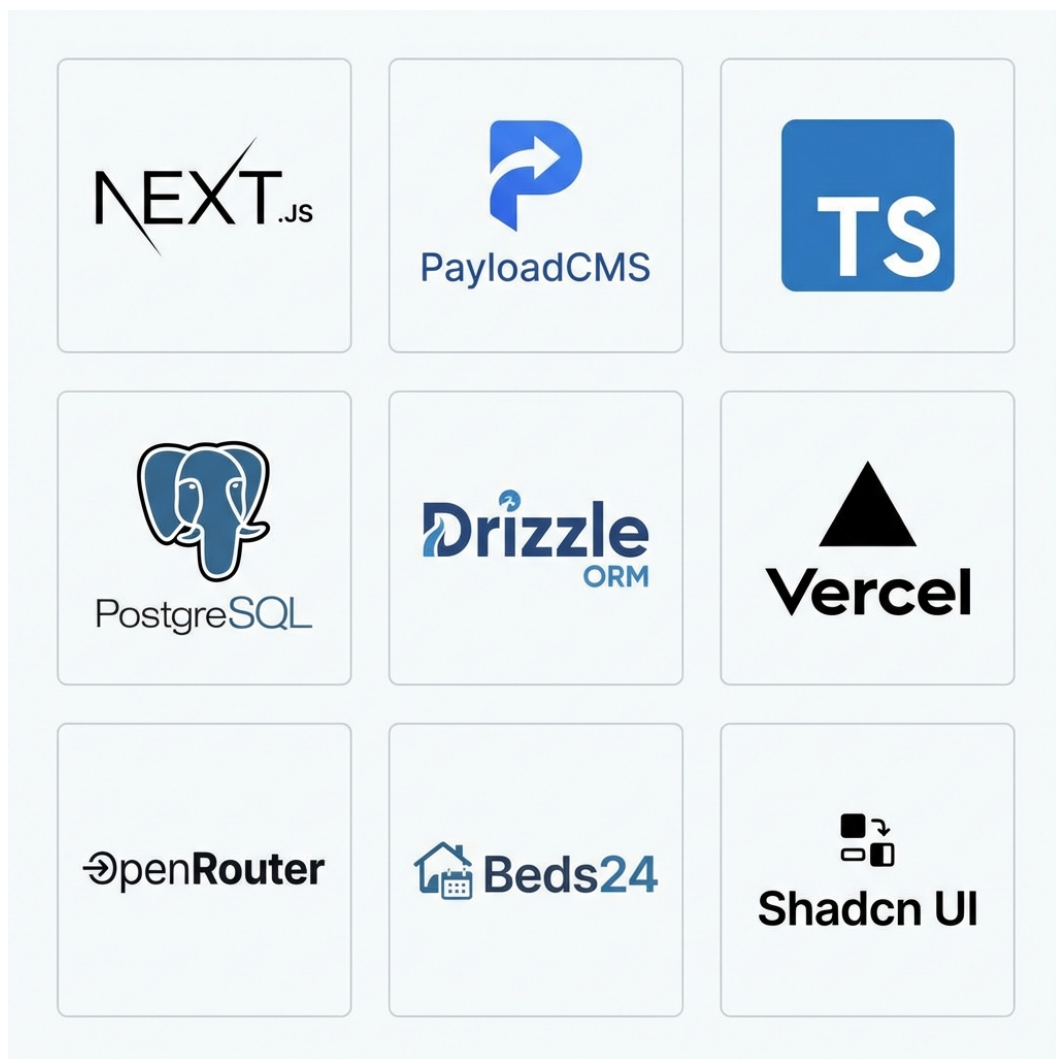
1.4 Συνεισφορά στο Rapid Development

Πέρα από την επιχειρησιακή της αξία, η εργασία αυτή αποτελεί μελέτη περίπτωσης για το πώς σύγχρονες τεχνολογίες (Next.js, PayloadCMS, Serverless) συμβάλλουν στο Rapid Application Development (RAD) [1]. Η δυνατότητα γρήγορης ανάπτυξης, εύκολου deployment (Serverless) [2] και άμεσης προσαρμογής (Code-First schema) επιτρέπει σε μικρές ομάδες ή μεμονωμένους developers να παράγουν λογισμικό επιχειρησιακού επιπέδου (Enterprise Grade) σε ελάχιστο χρόνο.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

2.1 Σύγχρονες Αρχιτεκτονικές Web



Σχήμα 2.1: Το τεχνολογικό stack της πλατφόρμας (Next.js, PayloadCMS, PostgreSQL, κ.α.).

2.1.1 TypeScript και Type Consistency

Η επιλογή της TypeScript έναντι της JavaScript αποτέλεσε μονόδρομο για την ανάπτυξη μιας σύγχρονης, κλιμακώσιμης εφαρμογής [3]. Η στατική τυποποίηση (static typing) που προσφέρει εξασφαλίζει «Type Consistency» σε όλο το εύρος της εφαρμογής (Fullstack). Αυτό σημαίνει ότι τα δεδομένα που επιστρέφει το Backend είναι εγγυημένα συμβατά με αυτά που περιμένει το Frontend, μειώνοντας δραματικά τα runtime errors και επιταχύνοντας την ανάπτυξη μέσω του IntelliSense.

2.1.2 PayloadCMS: Backend Framework vs CMS

Το PayloadCMS (ειδικά από την έκδοση 3.0 και μετά) διαφοροποιείται σημαντικά από τα παραδοσιακά CMS (όπως το WordPress ή το Strapi) [4]. Λειτουργεί πρωτίστως ως ένα Backend Framework που τρέχει εντός του Next.js, παρόμοια με το Django ή το Laravel, αλλά με τη δύναμη του TypeScript.

Code-First Collections: Οι δομές δεδομένων (Collections) ορίζονται αποκλειστικά μέσω κώδικα (TypeScript objects), λειτουργώντας ταυτόχρονα ως σχήμα βάσης δεδομένων, μοντέλο API και διεπαφή διαχείρισης.

Next.js Integration: Το backend (Admin panel, API routes, Auth) είναι ενσωματωμένο στην εφαρμογή Next.js, μετατρέποντάς την σε μια full-stack λύση χωρίς την ανάγκη ξεχωριστού server.

2.1.3 Next.js, Server Actions και Monorepo

Το Next.js επιλέχθηκε όχι μόνο για τις δυνατότητες Rendering (SSR, SSG) αλλά και για την άριστη συμβατότητά του με Serverless υποδομές [5, 6].

Server Actions: Μια κρίσιμη λειτουργία που επιτρέπει στο Frontend να καλεί συναρτήσεις του Backend απευθείας, χωρίς την ανάγκη ρητού ορισμού API Endpoints. Αυτό απλοποιεί τη διαχείριση δεδομένων και διατηρεί την τυποποίηση (types) από άκρη σε άκρη.

Monorepo: Η διατήρηση όλου του κώδικα (Frontend, Backend, Webhooks) σε ένα ενιαίο Repository επιτρέπει την κοινή χρήση τύπων και utilities, εξαιλείοντας την ανάγκη για συγχρονισμό μεταξύ διαφορετικών projects.

2.1.4 PostgreSQL και Drizzle ORM

Η πλατφόρμα βασίζεται στην PostgreSQL [7], μια ισχυρή σχεσιακή βάση δεδομένων. Η επικοινωνία με τη βάση γίνεται μέσω του Drizzle ORM [8], το οποίο είναι πλήρως συμβατό με το PayloadCMS.

Low-Level Capabilities: Το Drizzle επιτρέπει στον developer να γράψει σύνθετα SQL queries όταν οι abstraction layers του CMS δεν επαρκούν, προσφέροντας απόλυτο έλεγχο στην απόδοση.

Type Safety: Πλήρης τυποποίηση των queries βάσει του σχήματος της βάσης.

2.1.5 Shadcn UI, React και Zustand

Για το User Interface χρησιμοποιήθηκε το Shadcn UI [9], το οποίο διαφέρει από τις παραδοσιακές βιβλιοθήκες (όπως το Material UI) καθώς δεν είναι ένα ηχηρή package, αλλά κώδικας που αντιγράφεται μέσα στο project. Αυτό δίνει απόλυτη ελευθερία παραμετροποίησης.

React: Η βάση του frontend για τη δημιουργία επαναχρησιμοποιήσιμων components [10].

Zustand: Χρησιμοποιήθηκε για τη διαχείριση της κατάστασης (state management) της εφαρμογής [11], προσφέροντας μια πιο απλή και ελαφριά εναλλακτική στο Redux.

2.1.6 Swagger/OpenAPI και Code Generation

Για την επικοινωνία μεταξύ της Πλατφόρμας και των Websites, χρησιμοποιήθηκε το πρότυπο OpenAPI (Swagger) [12].

Generation: Η βιβλιοθήκη next-swagger-doc παράγει αυτόματα το documentation του API.

Client: Η βιβλιοθήκη openapi-typescript-codegen χρησιμοποιείται στο Website project για να παράγει έναν πλήρως τυποποιημένο client (SDK), εξασφαλίζοντας ότι το Website είναι πάντα συγχρονισμένο με τις αλλαγές στο API της πλατφόρμας.

2.2 Large Language Models (LLMs)

Η ενσωμάτωση LLMs (μέσω API όπως OpenAI ή Anthropic) μεταμορφώνει τις εφαρμογές από στατικά εργαλεία σε έξυπνους βοηθούς [13]. Στο πλαίσιο

του PMS, τα LLMs μπορούν να αναλύσουν το context μιας κράτησης και να προτείνουν απαντήσεις, να μεταφράσουν μηνύματα ή να λειτουργήσουν ως virtual concierges στις ιστοσελίδες των καταλυμάτων.

2.2.1 Vercel AI SDK και Streaming

Για την υλοποίηση των AI λειτουργιών χρησιμοποιήθηκε το **Vercel AI SDK (v5)** [14]. Το SDK αυτό επιλέχθηκε διότι απλοποιεί δραματικά τη διαδικασία του **Streaming** (ροή δεδομένων), επιτρέποντας στο UI να εμφανίζει την απάντηση του AI σταδιακά (λέξη-λέξη) καθώς παράγεται, βελτιώνοντας την εμπειρία χρήστη.

2.2.2 OpenRouter και Πρόσβαση σε Μοντέλα

Η πλατφόρμα δεν συνδέεται απευθείας με έναν πάροχο (π.χ. OpenAI), αλλά χρησιμοποιεί το **OpenRouter** [15]. Το OpenRouter λειτουργεί ως ένας ενδιαμέσος κόμβος (gateway) που παρέχει πρόσβαση σε πληθώρα μοντέλων (GPT-4, Claude 3.5 Sonnet, Llama 3, κ.α.) μέσω ενός ενιαίου API. Αυτό δίνει την ευελιξία στον χρήστη να επιλέξει το μοντέλο που ταιριάζει καλύτερα στις ανάγκες και το budget του, χωρίς αλλαγές στον κώδικα.

2.3 Channel Managers και Διασυνδεσιμότητα

Η επιλογή του **Beds24** [16] ως Channel Manager δεν ήταν τυχαία. Επιλέχθηκε διότι:

1. **Δημοφιλία & Κόστος:** Είναι μια από τις πιο διαδεδομένες και οικονομικές λύσεις στην αγορά [17].
2. **Modern API:** Διαθέτει ένα σύγχρονο, comprehensive API με υποστήριξη OpenAPI, το οποίο επιτρέπει την εύκολη παραγωγή κώδικα (μέσω swagger-typescript-api) και την άμεση προσαρμογή σε μελλοντικά updates.

Κεφάλαιο 3

Μεθοδολογία και Ανάλυση Απαιτήσεων

3.1 Ρόλοι και Ιεραρχία Χρηστών

Το σύστημα σχεδιάστηκε με γνώμονα την ιεραρχία Manager-Owner, υλοποιώντας ένα εύκαμπτο μοντέλο δικαιωμάτων που επιτρέπει τον έλεγχο σε κάθε επίπεδο.

3.1.1 Τύποι Χρηστών (User Types)

Η πλατφόρμα υποστηρίζει τέσσερις διακριτούς τύπους χρηστών, όπως παρουσιάζονται στον Πίνακα 3.1.

Πίνακας 3.1: Τύποι χρηστών της πλατφόρμας

User Type	Περιγραφή	Parent	Children
platform-user	Administrators πλατφόρμας	Κανένας	Managers
manager	Property Managers	Platform	Owners, Staff
owner	Ιδιοκτήτες ακινήτων	Manager	Staff
stuff	Προσωπικό	Owner/Manager	Κανένας

Platform Users (Admins):

- Πλήρης πρόσβαση σε όλα τα δεδομένα της πλατφόρμας (Admin View)
- Δημιουργία και διαχείριση Plans (πλάνα χρέωσης)
- Εποπτεία όλων των Managers, χωρίς να είναι «γονείς» τους ιεραρχικά, αλλά έχοντας δυνατότητα διαχείρισης.

Managers:

- Σύνδεση με Channel Manager (Beds24)
- Εισαγωγή και διαχείριση Properties
- Δημιουργία Owners και Staff

- Ορισμός δικαιωμάτων για τους υφισταμένους
- Δυνατότητα παραμετροποίησης των δικών τους access configurations
- Δημιουργία και διαχείριση Auto Actions
- Δημιουργία Websites για τα ακίνητα

Owners:

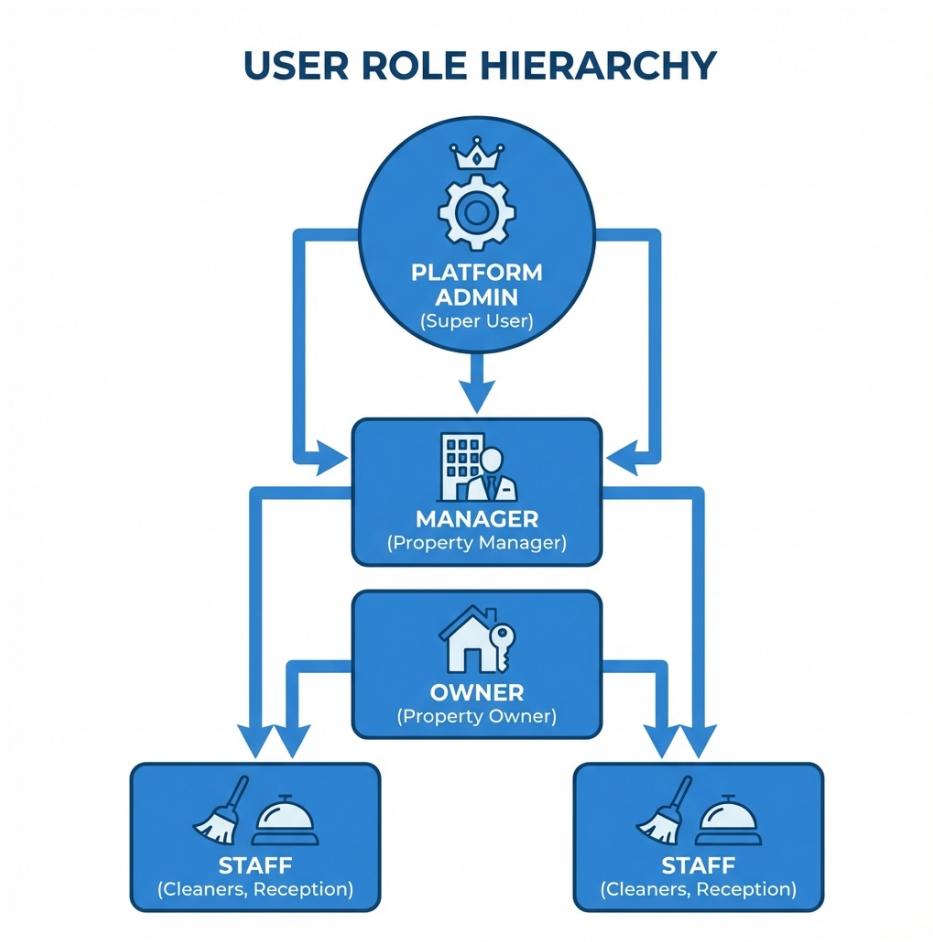
- Προβολή/επεξεργασία μόνο των ακινήτων που τους έχουν ανατεθεί
- Δικαιώματα καθορίζονται πλήρως από τον Manager
- Δημιουργία Staff που ανήκουν σε αυτούς

Staff:

- Ελάχιστα δικαιώματα (συνήθως view-only)
- Δεν μπορούν να δημιουργήσουν άλλους χρήστες
- Χρήσιμοι για εργασίες καθαρισμού, check-in, κ.λπ.

3.1.2 Ιεραρχικές Σχέσεις και Billing

Κάθε Owner ανήκει σε έναν Manager/Platform-User, ενώ το Staff μπορεί να ανήκει σε Manager ή Owner. Η σχέση αυτή καθορίζει την πρόσβαση και τη ροή των χρεώσεων. Κάθε Manager και Owner συνδέεται με ένα Plan που καθορίζει τις χρεώσεις και τα features.



Σχήμα 3.1: Ιεραρχική δομή ρόλων χρηστών στην πλατφόρμα.

3.2 Αρχιτεκτονική Συστήματος (Monorepo)

Η πλατφόρμα υλοποιείται ως Monorepo, το οποίο εξασφαλίζει συνοχή μεταξύ των διαφορετικών components και διευκολύνει την κοινή χρήση τύπων και utilities.

3.2.1 Δομή Monorepo

Listing 3.1: Δομή καταλόγων του Monorepo

```

1 lunarety-v2/|—
2   src/|—
3     app/           # Next.js App Router||—
4       (frontend)/  # Frontend routes||—
5       (payload)/   # PayloadCMS admin routes|—
6     apis/          # External API integrations||—
7       channel-managers/ # Beds24 API, DTOs, sync|—
8     collections/   # PayloadCMS collections|—
  
```

```
9     crons/                # Scheduled jobs|└─
10    endpoints/           # Custom API endpoints|└─
11    lib/                  # Shared utilities|└─
12    payload.config.ts     # PayloadCMS configuration|└─
13    lunarety-v2-website/  # Website template└─
14    package.json
```

3.2.2 Βασικά Components

PayloadCMS Core:

- Headless CMS με PostgreSQL backend
- Αυτόματη δημιουργία REST και GraphQL API
- Admin panel για διαχείριση δεδομένων
- Type-safe collection definitions

Next.js App Router:

- Server-side rendering για SEO
- Server Actions για data mutations
- Middleware για authentication
- Dynamic routing για Calendar, Messages, Settings

Channel Manager API Layer:

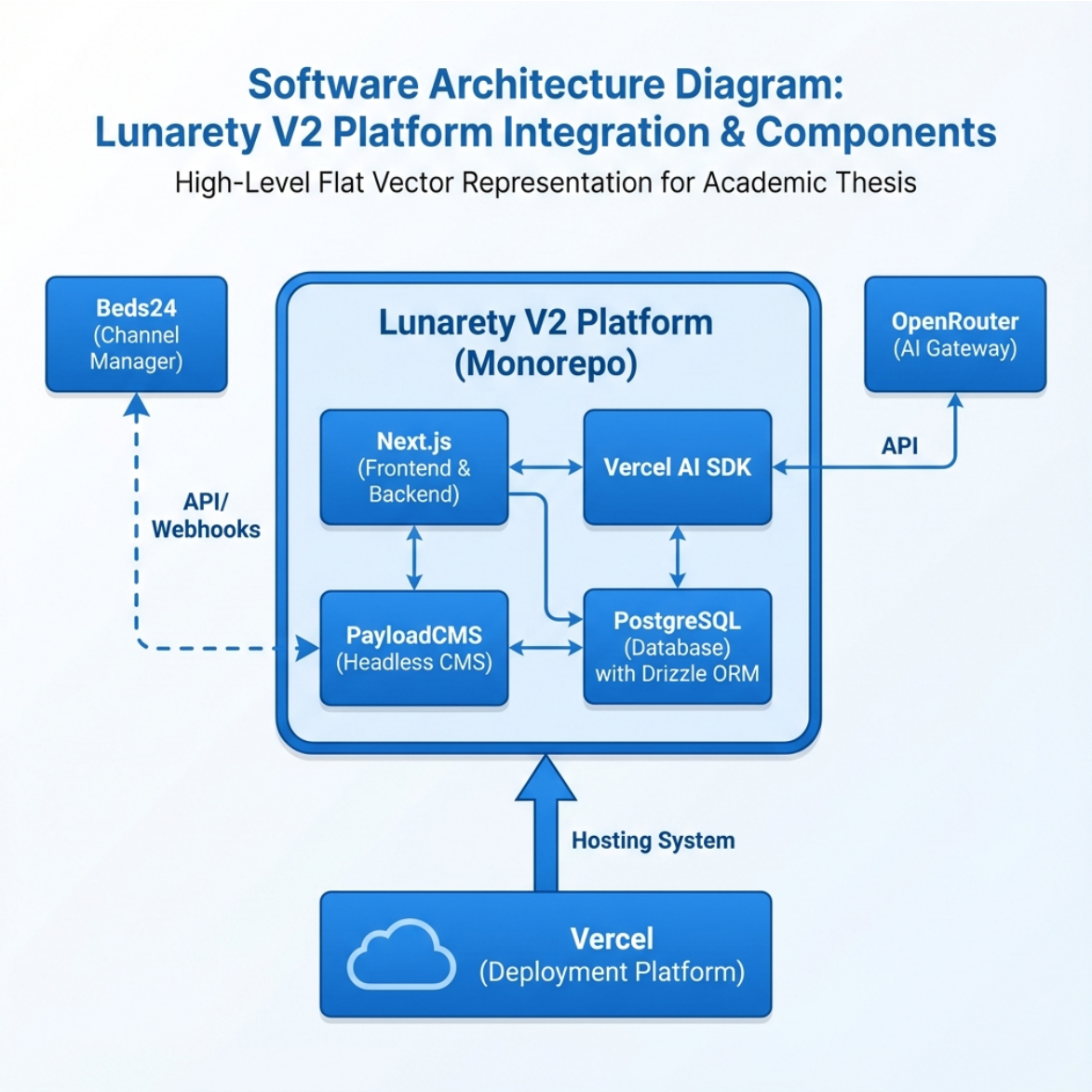
- ChannelManagerApi object – central export point
- DTOs για μετατροπή Beds24 ↔ Payload formats
- Abstraction layer για μελλοντική υποστήριξη άλλων Channel Managers

Το κρίσιμο σημείο αυτής της αρχιτεκτονικής είναι ότι ολόκληρη η πλατφόρμα χρησιμοποιεί αυτές τις συναρτήσεις **χωρίς να είναι εξαρτημένη από το Beds24**. Οι υπόλοιπες μονάδες της εφαρμογής (hooks, cron jobs, webhooks) καλούν τα generic methods του ChannelManagerApi object, τα οποία εσωτερικά μεταφράζονται στις αντίστοιχες κλήσεις του συγκεκριμένου Channel Manager. Αυτός ο σχεδιασμός ακολουθεί το πρότυπο **Adapter Pattern** [18, 19].

3.2.3 External Integrations

Πίνακας 3.2: External Services και Integrations

Service	Σκοπός	Module
Beds24 API	Channel Manager	apis/channel-managers/beds24/
Vercel AI SDK	AI text improvements	lib/ai/
OpenRouter	LLM API proxy	lib/ai/
Telegram Bot API	User notifications	lib/telegram/
Resend	Email sending	lib/email/



Σχήμα 3.2: Διάγραμμα αρχιτεκτονικής συστήματος (Monorepo).

Παράλληλα, το lunarety-v2-website αποτελεί ένα ξεχωριστό public project στο GitHub, σχεδιασμένο να λειτουργεί ως template για τις ιστοσελίδες των

καταλυμάτων. Επικοινωνεί με την πλατφόρμα μέσω ενός secure API endpoint χρησιμοποιώντας WEBSITE_API_KEY.

3.3 Μοντέλο Δεδομένων και Σχέσεις

3.3.1 Κύριες Οντότητες (Collections)

Η βάση δεδομένων (PostgreSQL) οργανώνεται γύρω από τις κύριες collections που παρουσιάζονται στον Πίνακα 3.3.

Πίνακας 3.3: Κύριες Collections της βάσης δεδομένων

Collection	Περιγραφή	Σημαντικά Fields
users	Χρήστες	type, managedBy, ownedBy,
property	πλατφόρμας	settings
	Καταλύματα	manager, owner,
bookings	Κρατήσεις	channelPropertyId, rooms
		property, resource, pricing,
		messages
property-	Τιμές ανά ημέρα	property, date, channelRoomId,
rates		availability
billing	Χρεώσεις/Τιμολόγια	type, ownerDebtor,
		managerCreditor
auto-	Κανόνες	type, triggerType, template
actions	αυτοματισμού	
plans	Πλάνα χρέωσης	planCommission, features
website	Generated	apiKey, type, properties
	websites	

3.3.2 Σχέσεις Μεταξύ Οντοτήτων

Property ↔ Users:

- Κάθε Property έχει έναν manager (υποχρεωτικό)
- Κάθε Property μπορεί να έχει έναν owner (προαιρετικό)

Bookings ↔ Property ↔ Users:

- Κάθε Booking ανήκει σε ένα Property
- Το Access control βασίζεται στη σχέση Property → Manager/Owner

Property Rates ↔ Property ↔ Bookings:

- Κάθε Rate συνδέεται με μια ημέρα και ένα δωμάτιο
- Τα Bookings συνδέονται με τα rates μέσω relatedRates
- Join field bookingsStayingOnThisNight για reverse lookup

Auto Actions ↔ Booking ↔ Users/Properties:

- Κάθε Auto Action ενεργοποιείται (Trigger) από γεγονότα που αφορούν ένα **Booking**
- Συνδέει και χρησιμοποιεί δεδομένα από πολλά Properties και Users
- Τα Properties μπορούν να «εγγραφούν» (subscribe) σε Auto Actions

3.3.3 Hook System και Lifecycle

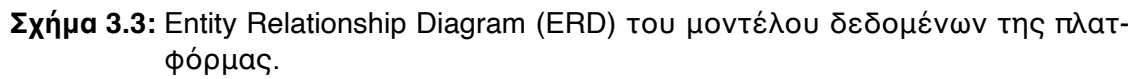
Το PayloadCMS παρέχει ένα ισχυρό σύστημα hooks που χρησιμοποιείται εκτενώς (Πίνακας 3.4).

Πίνακας 3.4: Hook Types και χρήση στην πλατφόρμα

Hook Type	Εκτελείται	Χρήση
beforeChange	Πριν την αποθήκευση	Sync με CM, Commission calc
afterChange	Μετά την αποθήκευση	Touch rates, Trigger actions
beforeRead	Πριν την ανάγνωση	Data transformation
afterRead	Μετά την ανάγνωση	Computed fields, formatting

Παράδειγμα Hook Chain για Booking Creation:

1. linkRatesToBooking (beforeChange) – Σύνδεση με property rates
2. syncChannelBooking_beforeChangeHook (beforeChange) – Υπολογισμός commissions, sync με Beds24
3. touchNightlyRates (afterChange) – Ενημέρωση σχετικών rates



Σχήμα 3.3: Entity Relationship Diagram (ERD) του μοντέλου δεδομένων της πλατφόρμας.

Κεφάλαιο 4

Υλοποίηση

4.1 Διαχείριση Κρατήσεων και Συγχρονισμός

Η διαχείριση κρατήσεων αποτελεί τον πυρήνα της πλατφόρμας και απαιτεί προσεκτικό σχεδιασμό για να εξασφαλιστεί η ακεραιότητα των δεδομένων, η απόδοση και η συνέπεια μεταξύ της τοπικής βάσης δεδομένων και του Channel Manager.

4.1.1 Αρχική Σύνδεση και Ρύθμιση Webhooks

Η διασύνδεση με το Beds24 ακολουθεί μια συγκεκριμένη ροή αρχικοποίησης:

1. **Σύνδεση Λογαριασμού:** Ο Manager εισάγει το Refresh Token του Beds24 στις ρυθμίσεις του λογαριασμού του.
2. **Token Exchange:** Το σύστημα αυτόματα ανταλλάσσει το Refresh Token για ένα Access Token μέσω του `b24ApiRefreshToken()`.
3. **User ID Retrieval:** Ανακτάται το `channelManagerUserId` από το Beds24.
4. **Initial Property Import:** Εκτελείται μαζική εισαγωγή όλων των Properties και των Property Rates.
5. **Webhook Configuration:** Για κάθε Property, ρυθμίζεται αυτόματα το Beds24 να στέλνει Webhooks.

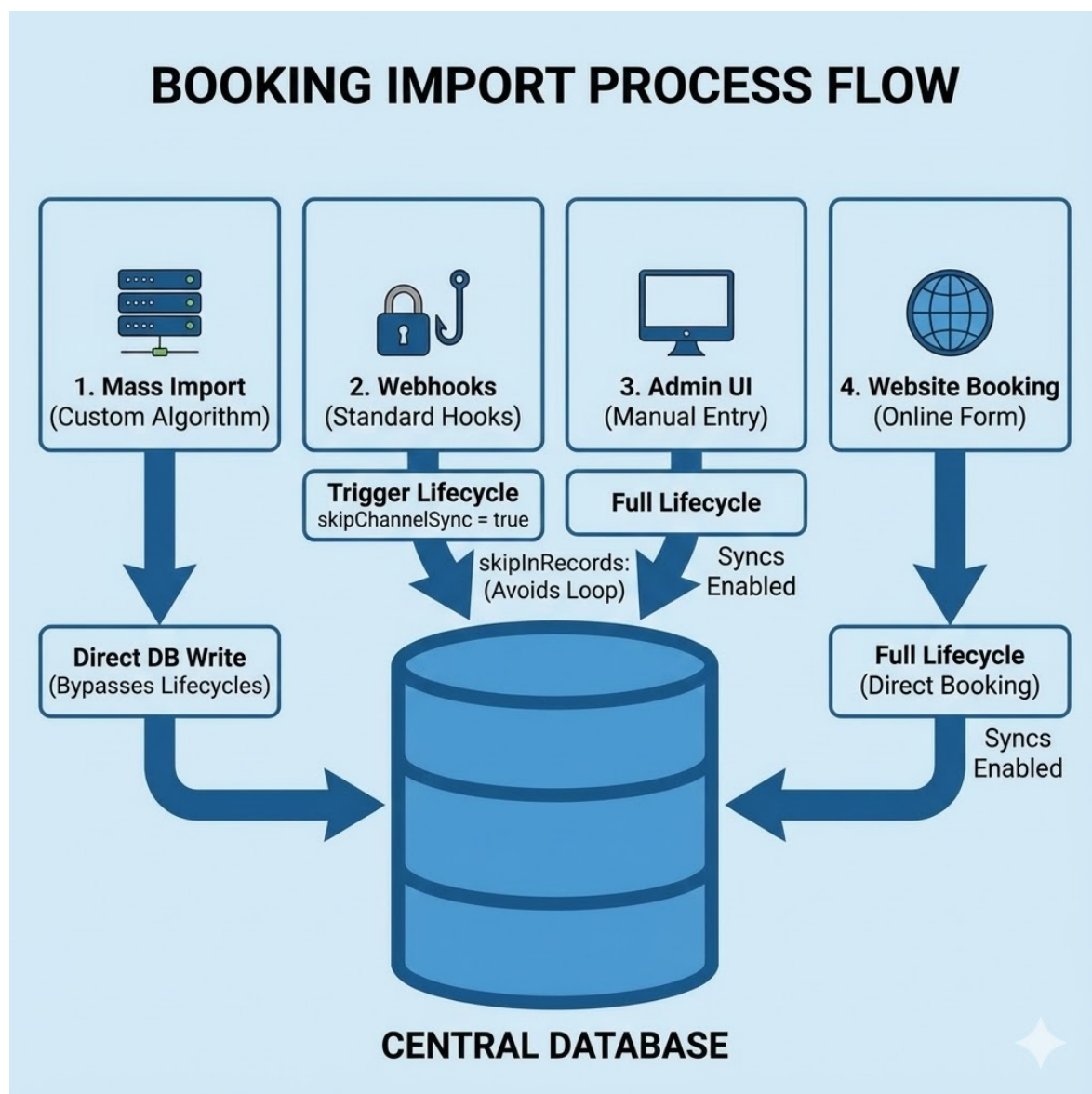
Listing 4.1: Αυτόματη ρύθμιση webhooks κατά την εισαγωγή properties

```
1 // Automatic webhook setup during property import
2 await api.properties.propertiesCreate(
3   propertiesWithRates.map(({ convertedProperty }) => ({
4     id: Number(convertedProperty.channelPropertyId),
5     webhooks: {
6       url:
         `${process.env.NEXT_PUBLIC_URL}/api/webhooks/channel-managers/beds24/bookings`,
```

```
7     customHeader: 'secretPropertyKey: ${convertedProperty.secretPropertyKey}',  
8     version: "twoWithPersonalData",  
9     additionalData: "none",  
10  },  
11  })))  
12 );
```

4.1.2 Μέθοδοι Εισαγωγής Κρατήσεων

Η πλατφόρμα υποστηρίζει τέσσερις διαφορετικές μεθόδους δημιουργίας/ενημέρωσης κρατήσεων:



Σχήμα 4.1: Μέθοδοι εισαγωγής κρατήσεων στην πλατφόρμα.

4.1.2.1 A) Mass Import (Property Syncing)

Κατά την αρχική εισαγωγή ή τον περιοδικό συγχρονισμό, η πλατφόρμα χρειάζεται να εισάγει εκατοντάδες ή χιλιάδες κρατήσεις. Χρησιμοποιείται η `payload.db.create()` και `payload.db.updateOne()` αντί των `standard methods`, παρακάμπτοντας τα `hooks` για βέλτιστη απόδοση.

Listing 4.2: Mass import με direct DB operations

```
1 const syncBookingsOperations = async (
2   bookingsToCreate: RequiredDataFromCollectionSlug<"bookings">[],
3   bookingsToUpdate: Array<{
4     id: number;
5     data: RequiredDataFromCollectionSlug<"bookings">;
6   }>,
7   bookingsToDelete: Booking[]
8 ) => {
9   // CREATE - Direct DB, bypassing lifecycle
10  for (const booking of bookingsToCreate) {
11    await payload.db.create({
12      collection: "bookings",
13      data: dtoPayloadToPayloadDb(booking),
14    });
15  }
16
17  // UPDATE - Direct DB, bypassing lifecycle
18  for (const { id, data } of bookingsToUpdate) {
19    await payload.db.updateOne({
20      collection: "bookings",
21      id: id,
22      data: dtoPayloadToPayloadDb(data),
23    });
24  }
25 };
```

4.1.2.2 B) Webhook Updates (Channel Manager Triggers)

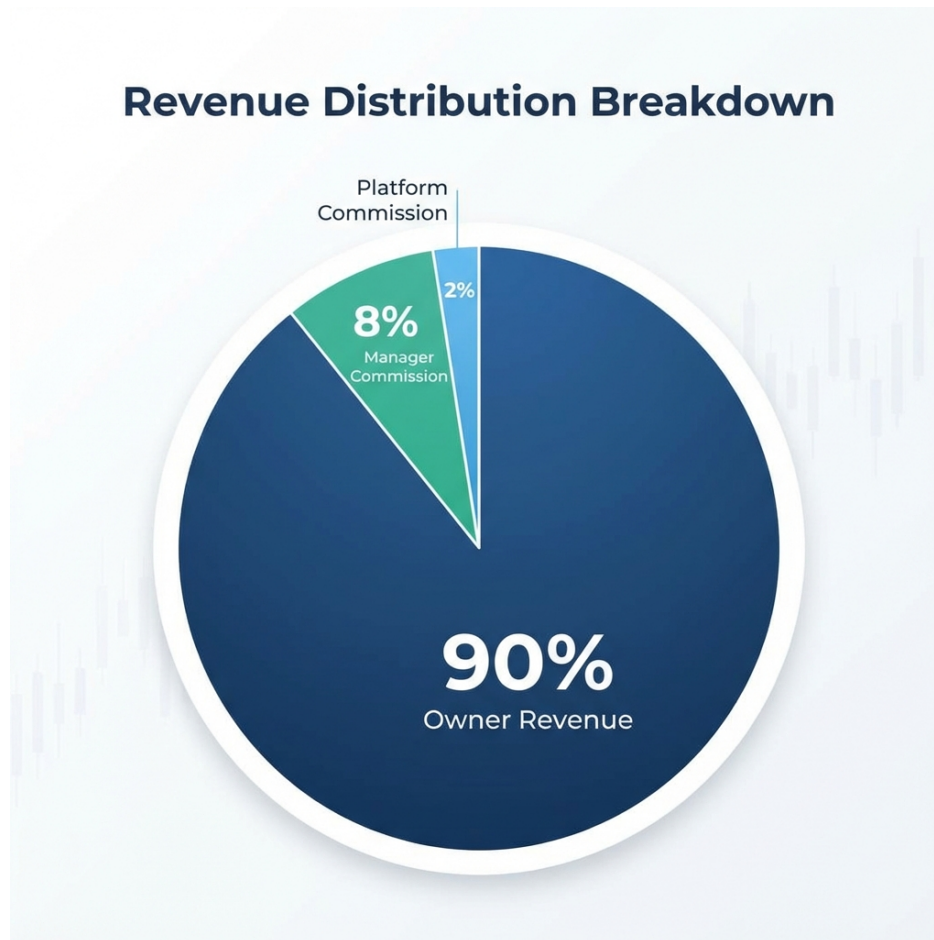
Όταν μια κράτηση δημιουργείται ή ενημερώνεται στο Beds24, η πλατφόρμα λαμβάνει ένα `webhook notification`. Κρίσιμο είναι το `skipChannelSync` flag:

Listing 4.3: Αποτροπή infinite loop με `skipChannelSync` flag

```
1 const createBookingRes = await payload.create({
2   collection: "bookings",
3   data: { ...dtoPayloadToPayloadDb(newBooking), skipChannelSync: true },
4   req: { transactionID: transactionId },
5 });
```

4.1.2.3 Γ) UI-Based Booking Creation

Όταν ένας χρήστης δημιουργεί κράτηση μέσω του UI, εκτελείται το πλήρες lifecycle συμπεριλαμβανομένου του υπολογισμού προμηθειών:



Σχήμα 4.2: Τριπλή δομή προμηθειών (Three-Tier Commission Structure).

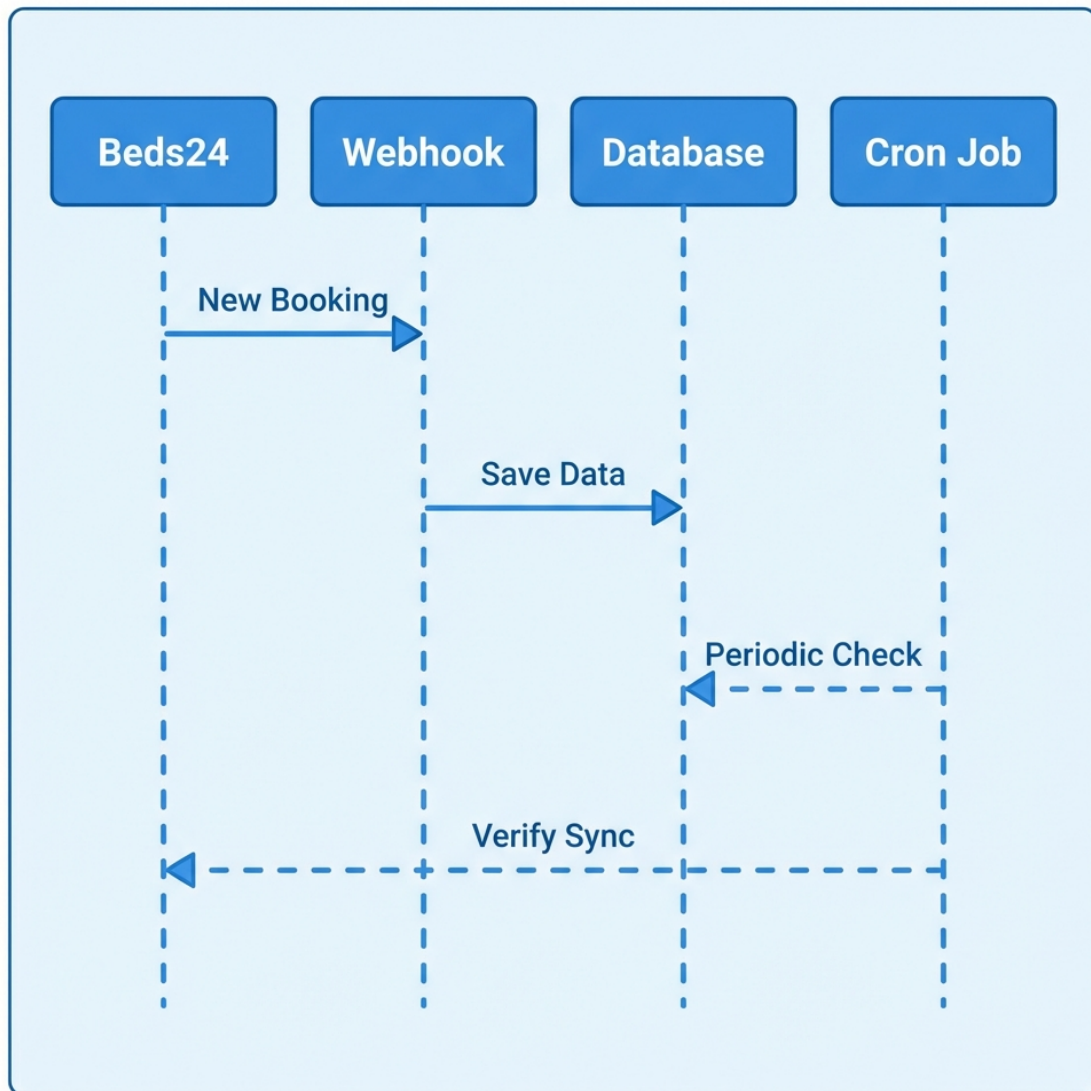
Listing 4.4: Υπολογισμός commission

```
1 export const computeCommissionOfPlan = (  
2   booking: Partial<Booking>,  
3   plan: Plan  
4 ): number => {  
5   const bookingTotal = booking.pricing?.totalAmount ?? 0;  
6  
7   if (booking.excludedFromCommission) return 0;  
8  
9   if (plan.planCommission?.commissionType === "percentage") {  
10    const percentage = plan.planCommission.commissionPercentage ?? 0;  
11    return Math.round(bookingTotal * (percentage / 100) * 100) / 100;  
12  } else {  
13    return plan.planCommission?.fixedCommission ?? 0;  
14  }
```

```
15 };
```

4.1.2.4 Δ) Website Booking Creation

Για κρατήσεις από το generated website, το πεδίο source ορίζεται αντίστοιχα (websitePlatform, websiteManager, websiteOwner).



Σχήμα 4.3: Ροή δεδομένων συγχρονισμού κρατήσεων με το Beds24.

4.2 Μηχανισμός Auto Actions

Το σύστημα Auto Actions είναι η «καρδιά» του αυτοματισμού της πλατφόρμας.

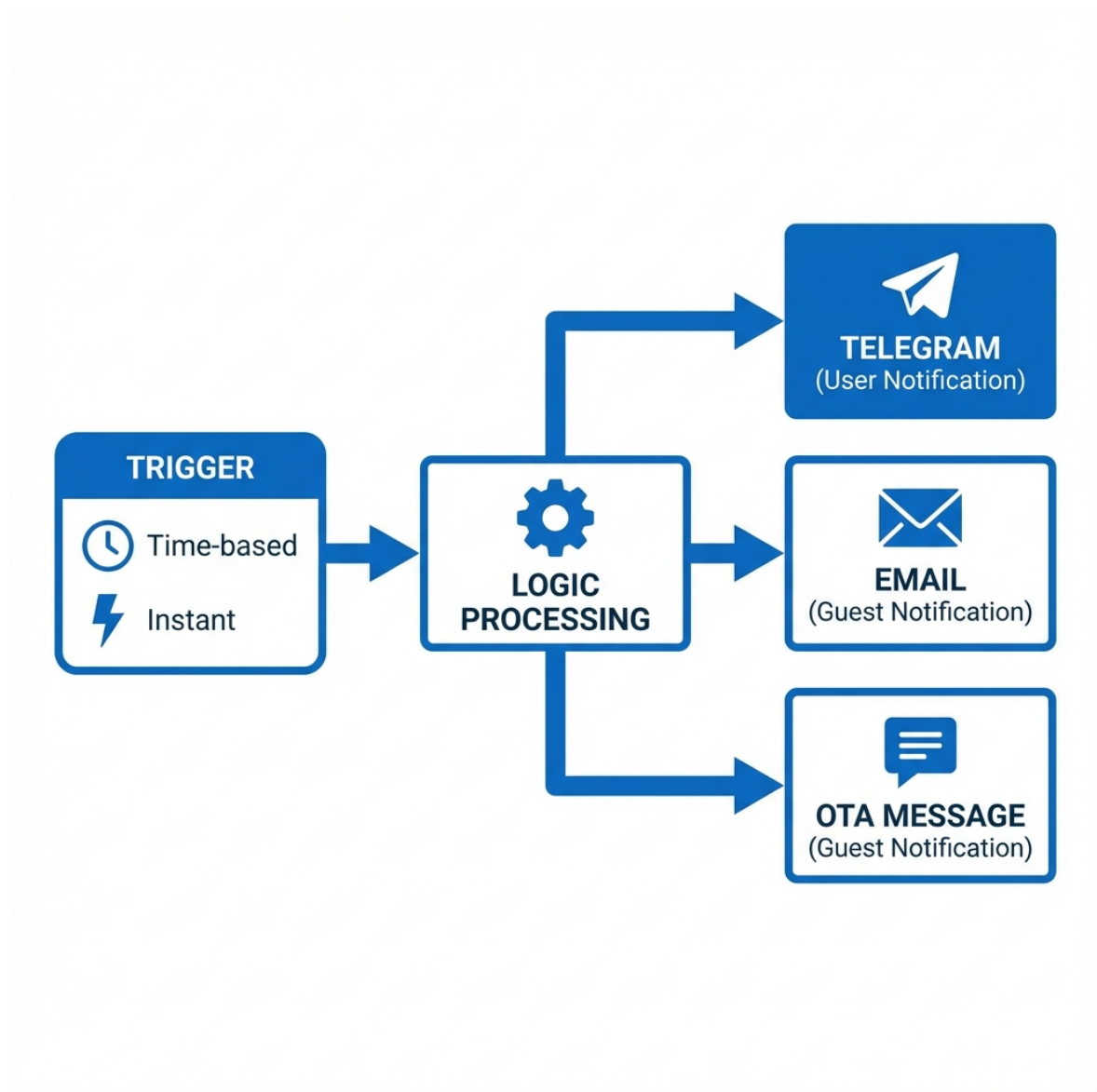
4.2.1 Κατηγορίες Auto Actions

Υπάρχουν τρεις διακριτοί τύποι:

1. **Notifications to Users:** Ειδοποιήσεις προς τους χρήστες της πλατφόρμας (Managers, Owners, Staff).
2. **Property Notifications to Guests:** Μηνύματα προς τους επισκέπτες (π.χ. οδηγίες άφιξης).
3. **Simple Template:** Πρότυπα κειμένου χωρίς trigger condition, για χειροκίνητη αποστολή μέσω Chat.

4.2.2 Κανάλια Επικοινωνίας

- **Για Guests:** OTA Message (μέσω API καναλιού) ή Email.
- **Για Users:** Telegram. Η αρχιτεκτονική επιτρέπει εύκολη προσθήκη νέων καναλιών (WhatsApp, SMS).



Σχήμα 4.4: Διάγραμμα ροής εκτέλεσης των Auto Actions.

4.2.3 Μηχανισμός Προτύπων (Template Engine)

Αναπτύχθηκε custom Template Engine με τις εξής δυνατότητες:

Variable Replacement: Σύνταξη `[object.property]` για πρόσβαση σε εμφωλευμένα δεδομένα

Conditional Logic: Σύνταξη `{condition?? result}` για εμφάνιση κειμένου υπό συνθήκη

Date Formatting: Αυτόματη αναγνώριση και μορφοποίηση ημερομηνιών

Listing 4.5: Παράδειγμα Template για νέα κράτηση

```

1  Νέα κράτηση
2  : []Επιβεβαιωμένη κράτηση
3    - [property.title] []
4
5  [booking.bookingHolder.firstName] [booking.bookingHolder.lastName] []
6  [booking.resource.adults] Ενήλικες, [booking.resource.children] Παιδιά [] Από
7    [booking.resource.checkIn] έως [booking.resource.checkOut]
8
9  {user.settings.features.bookings.prices !== "none" ?? Τιμή
10   : €[booking.pricing.totalAmount]}
11
12 {user.settings.features.messages.visibility !== "none" ?? Μηνύματα
13   : [link-booking-chat]}

```

4.2.4 Time-based Triggers (Cron Jobs)

Πίνακας 4.1: Υποστηριζόμενοι Time-based Trigger Types

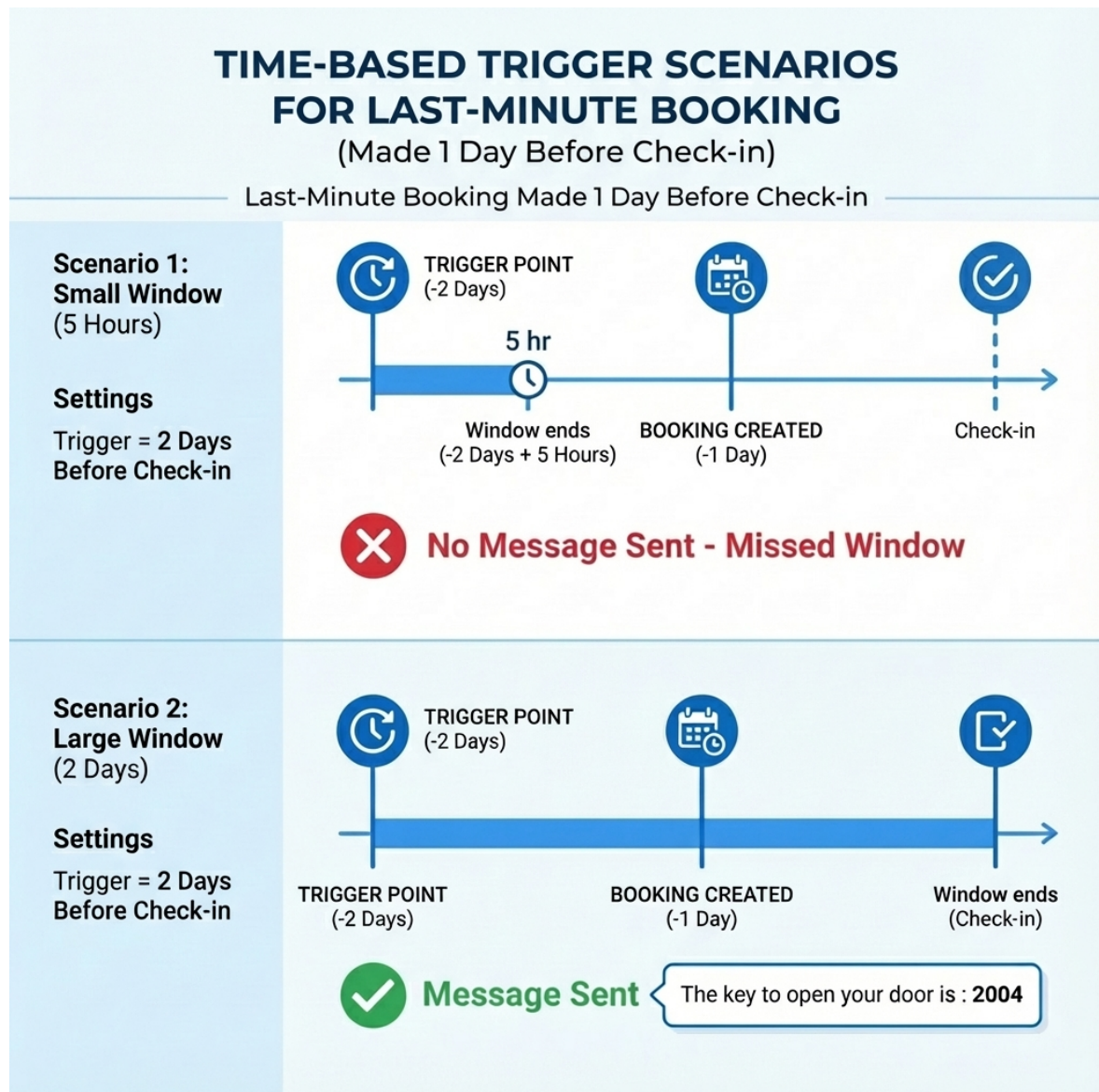
Trigger Type	Περιγραφή	Πεδίο Κράτησης
before-checkin	Πριν την άφιξη	resource.checkInStartOfDay
after-checkout	Μετά την αναχώρηση	resource.checkOutStartOfDay
after-cancellation	Μετά την ακύρωση	cancellationDate
after-new-booking	Μετά τη νέα κράτηση	reservationDate

Listing 4.6: Αλγόριθμος υπολογισμού χρονικού παραθύρου

```

1  const now: Date = new Date();
2  let fromDate: Date;
3  let toDate: Date;
4
5  if (triggerType.includes("before")) {
6    // before: [now + interval, now + interval + window]
7    fromDate = addHours(now, autoAction.timeInterval);
8    toDate = addHours(now, autoAction.timeInterval + autoAction.timeWindow);
9  } else if (triggerType.includes("after")) {
10   // after: [now - interval - window, now - interval]
11   fromDate = subHours(now, autoAction.timeInterval + autoAction.timeWindow);
12   toDate = subHours(now, autoAction.timeInterval);
13 }

```

Σχήμα 4.5: Σενάρια λειτουργίας Time-based Triggers για κράτηση τελευταίας στιγμής.

4.2.5 Αρχιτεκτονική Cron Jobs

Πίνακας 4.2: Scheduled Cron Jobs της πλατφόρμας

Cron Job	Συχνότητα	Αρχείο	Σκοπός
everyHourCron	Κάθε ώρα	everyHourCron.ts	Time-based Auto Actions
everyNightCron	Κάθε βράδυ	everyNightCron.ts	Token refresh, Full sync
everyMonthCron	Αρχή μήνα	everyMonthCron.ts	Billing records

4.3 Σύστημα Χρέωσης και Τιμολόγησης

4.3.1 Αρχιτεκτονική Χρέωσης

Το σύστημα billing δημιουργεί χρεώσεις ανά χρήστη στο τέλος κάθε μήνα:

- **Owner Bill:** Ο Owner (Debtor) οφείλει στον Manager (Creditor)
- **Manager Bill:** Ο Manager (Debtor) οφείλει στην Πλατφόρμα (Creditor)

4.3.2 Ανάλυση Χρεώσεων

Πίνακας 4.3: Τύποι χρεώσεων Billing

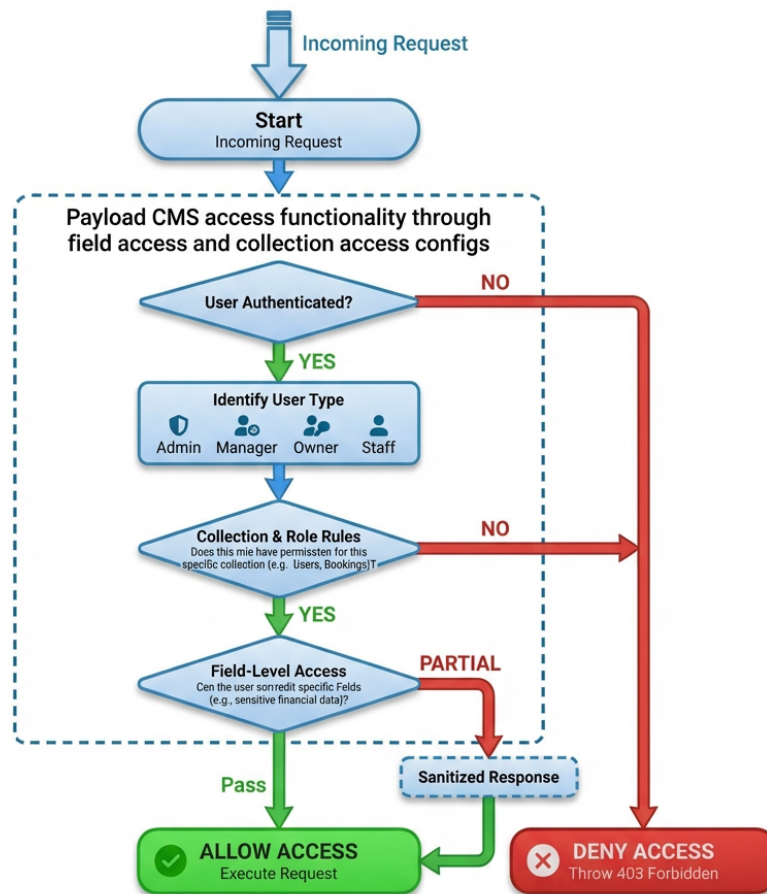
Τύπος	Περιγραφή	Υπολογισμός
propertyCommission	Ανά ακίνητο	$\text{count} \times \text{pricePerProperty}$
unitCommission	Ανά δωμάτιο	$\text{count} \times \text{pricePerUnit}$
bookingsCommission	Προμήθεια κρατήσεων	$\sum \text{commissions}$

4.3.3 Σύστημα Ελέγχου Πρόσβασης

Η πλατφόρμα υλοποιεί έλεγχο πρόσβασης σε δύο άξονες:

1. **Ρόλος Χρήστη:** platform-user, manager, owner, staff
2. **Χαρακτηριστικά Πλάνου:** Ρυθμίσεις από τον Manager για κάθε Owner/Staff

Access Control Decision Tree



Σχήμα 4.6: Decision tree για τον έλεγχο πρόσβασης.

4.4 Αυτοματοποιημένη Παραγωγή Ιστοσελίδων

4.4.1 Δομή και Λειτουργία Website

Το παραγόμενο website ακολουθεί βελτιστοποιημένη δομή τριών σελίδων:

1. **Home Page (/):** Multi-room search, property cards, filtering
2. **Property Page (/properties/[id]):** Gallery, description, AI Chat Bubble
3. **Booking Page (/booking/[id]):** Λεπτομέρειες κράτησης, self-service

4.4.2 Website API Endpoints

Πίνακας 4.4: Website API Endpoints

Endpoint	Method	Σκοπός
/api/website/properties	GET	Λίστα καταλυμάτων
/api/website/properties/[id]	GET	Λεπτομέρειες
/api/website/availability	POST	Έλεγχος διαθεσιμότητας
/api/website/booking	POST	Δημιουργία κράτησης
/api/website/booking/[id]	GET	Λεπτομέρειες κράτησης

4.5 Περιβάλλον Χρήστη (Platform UI)

4.5.1 Dashboard και Calendar

Το ημερολόγιο σχεδιάστηκε ως «Easy Calendar» με:

- **Drag and Drop:** Μετακίνηση κρατήσεων με τη βιβλιοθήκη @dnd-kit [20]
- **Infinite Scrolling:** Οριζόντια (ημερομηνίες) και κάθετα (properties)
- **Επιλογή Εύρους:** Μαζική αλλαγή τιμών και διαθεσιμότητας
- **Φίλτρα Properties:** Επιλογή συγκεκριμένων καταλυμάτων

4.5.2 Σύστημα Μηνυμάτων

Το Chat υποστηρίζει:

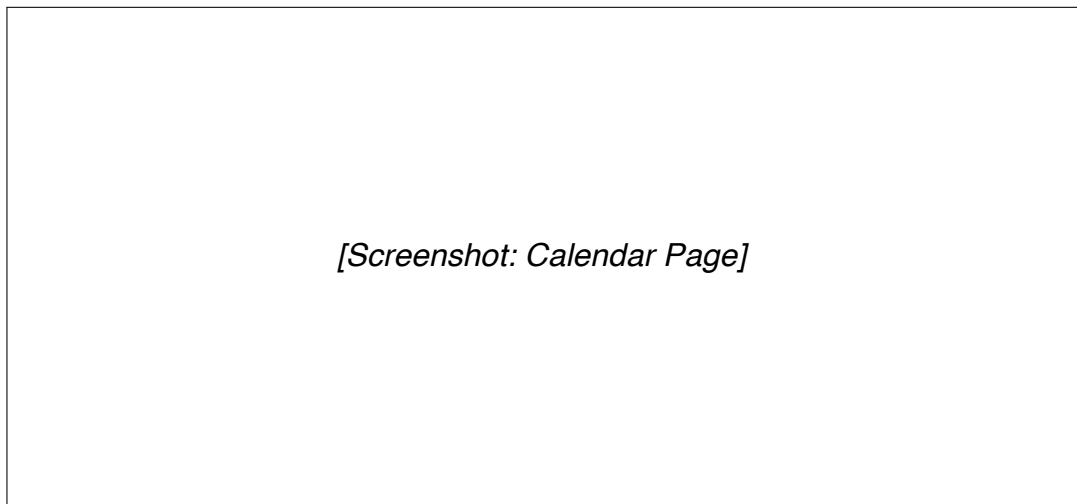
- Άμεση επικοινωνία με φιλοξενούμενους μέσω OTAs
- Χρήση Simple Templates με auto-fill
- Βελτίωση κειμένου μέσω AI («Improve with AI»)
- Infinite scrolling και αναζήτηση

Κεφάλαιο 5

Αποτελέσματα

5.1 Παρουσίαση της Πλατφόρμας

Η πλατφόρμα προσφέρει ένα μοντέρνο, responsive περιβάλλον εργασίας. Τα παρακάτω screenshots παρουσιάζουν την οπτική γωνία ενός χρήστη με πλήρη δικαιώματα (Admin/Manager).

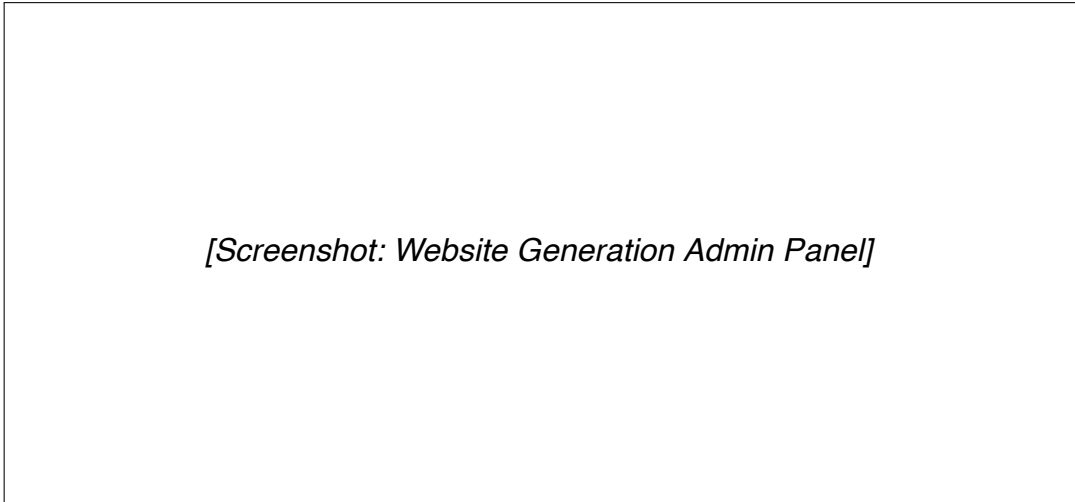


Σχήμα 5.1: Η σελίδα του Ημερολογίου (Calendar).



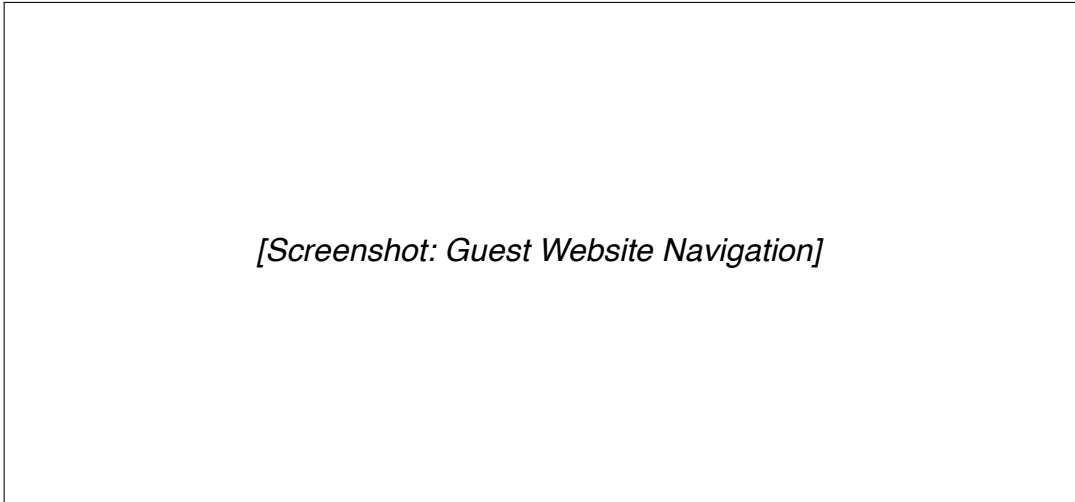
Σχήμα 5.2: Η σελίδα των Μηνυμάτων με AI capabilities.

5.2 Παρουσίαση του Generated Website



[Screenshot: Website Generation Admin Panel]

Σχήμα 5.3: Διαδικασία παραγωγής Website από το περιβάλλον του διαχειριστή.



[Screenshot: Guest Website Navigation]

Σχήμα 5.4: Πλοήγηση επισκέπτη στην παραγόμενη ιστοσελίδα.

[Screenshot: AI & LLM Capabilities]

Σχήμα 5.5: Δυνατότητες AI και LLM για την εξυπηρέτηση επισκεπτών.

5.3 Τεχνικές Προκλήσεις και Λύσεις

5.3.1 Συγχρονισμός Δεδομένων με External APIs

Πρόβλημα: Διατήρηση συνέπειας μεταξύ τοπικής βάσης και Beds24.

Λύση: Υιοθέτηση στρατηγικής όπου το Beds24 είναι η κύρια πηγή αλήθειας για δεδομένα κρατήσεων, με merge logic για internal fields.

5.3.2 Αποφυγή Infinite Loops

Πρόβλημα: Two-way sync δημιουργεί κύκλους.

Λύση: Χρήση skipChannelSync flag για αποτροπή reverse-update.

5.3.3 Απόδοση Μαζικών Εισαγωγών

Πρόβλημα: Εισαγωγή 60.000 records με standard lifecycle απαιτεί ώρες.

Λύση: Hybrid strategy με direct DB operations για bulk, standard lifecycle για μεμονωμένες αλλαγές.

Πίνακας 5.1: Σύγκριση στρατηγικών εισαγωγής

Περίπτωση	Μέθοδος	Πλεονέκτημα
Mass Property Import	Direct DB	50× ταχύτερο
Mass Rate Import	Bulk Insert	1 query για χιλιάδες records
Individual Updates	Standard Lifecycle	Consistency, hooks

Κεφάλαιο 6

Συζήτηση

6.1 Αξιολόγηση Τεχνολογικών Επιλογών

Η υιοθέτηση του μοντέλου «Backend Framework» με το PayloadCMS v3 και η χρήση Serverless υποδομών απέδειξε ότι είναι δυνατή η ανάπτυξη πολύπλοκων εφαρμογών με ελάχιστους πόρους.

Πίνακας 6.1: Αξιολόγηση τεχνολογικών επιλογών

Επιλογή	Όφελος	Trade-off
PayloadCMS v3	Rapid development, type-safety	Lock-in σε ecosystem
PostgreSQL	ACID transactions, complex queries	Setup complexity
Serverless Next.js App Router	Zero DevOps, auto-scaling SSR, Server Actions	Cold starts Learning curve

6.2 Μελλοντικές Βελτιώσεις

6.2.1 Βελτίωση UX στο Admin Panel

- GUI Builder για Templates με drag-and-drop
- Visual Auto Action Editor με flow-chart style
- Wizard-style user creation

6.2.2 Επέκταση Channel Manager Support

- Hostaway (υψηλή προτεραιότητα)
- Lodgify (μεσαία προτεραιότητα)
- Custom/Direct integrations

6.2.3 Mobile Application

- Push notifications για νέες κρατήσεις
- Quick actions (approve, respond)
- Offline calendar viewing

6.2.4 Advanced Analytics Dashboard

- Revenue per property/room
- Occupancy rates over time
- Booking source distribution

Κεφάλαιο 7

Συμπεράσματα

7.1 Επίτευξη Στόχων

Η πλατφόρμα Lunarety V2 επιτυγχάνει τον στόχο της να γεφυρώσει το χάσμα μεταξύ Channel Managers και ιδιοκτητών ακινήτων.

Πίνακας 7.1: Κύρια επιτεύγματα της πλατφόρμας

Στόχος	Υλοποίηση	Αποτέλεσμα
Two-way sync	Webhooks + API integration	Real-time consistency
Αυτοματοποίηση	Auto Actions (time/instant)	80% μείωση εργασιών
Ιεραρχία χρηστών	4-level system	Ανεξαρτησία διαχείρισης
Generated websites	One-click Vercel deploy	2-min deployment
AI integration	OpenRouter + Vercel AI SDK	24/7 ψηφιακή εξυπηρέτηση

7.2 Τεχνικά Συμπεράσματα

Αρχιτεκτονικές αποφάσεις που δικαιώθηκαν:

- PayloadCMS v3:** Το hook system αποδείχθηκε ιδανικό για complex business logic.
- PostgreSQL vs MongoDB:** Οι complex queries για billing και access control θα ήταν δυσκολότερες με document-based storage.
- Serverless Architecture:** Απλοποίησε το deployment και μείωσε τα operation costs.
- Monorepo Structure:** Διευκόλυνε τη συντήρηση και την κοινή χρήση τύπων.

7.3 Βασικά Διδάγματα

1. **Direct DB Operations για Mass Import:** Η παράκαμψη του lifecycle για bulk operations είναι απαραίτητη.
2. **Flag-based Loop Prevention:** Απλή και αποτελεσματική λύση για two-way sync.
3. **Feature-based Access Control:** Επιτρέπει παραμετροποίηση ανά Manager.
4. **AI as Enhancement:** Βελτιώνει την εμπειρία χωρίς να αντικαθιστά ανθρώπινη κρίση.

Βιβλιογραφία

- [1] James Martin. “Rapid Application Development”. Στο: *Macmillan Publishing* (1991).
- [2] Vercel. *Vercel Deployment Documentation*. 2024. URL: <https://vercel.com/docs> (επίσκεψη 01/11/2024).
- [3] Boris Cherny. *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media, 2019. ISBN: 978-1492037651.
- [4] PayloadCMS. *PayloadCMS Documentation*. 2024. URL: <https://payloadcms.com/docs> (επίσκεψη 01/11/2024).
- [5] Vercel. *Next.js Documentation*. 2024. URL: <https://nextjs.org/docs> (επίσκεψη 01/11/2024).
- [6] Eric Jonas κ.ά. “Cloud Programming Simplified: A Berkeley View on Serverless Computing”. Στο: *Technical Report No. UCB/EECS-2019-3* (2019).
- [7] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: <https://www.postgresql.org/docs/> (επίσκεψη 01/11/2024).
- [8] Drizzle Team. *Drizzle ORM Documentation*. 2024. URL: <https://orm.drizzle.team/> (επίσκεψη 01/11/2024).
- [9] shadcn. *Shadcn UI Documentation*. 2024. URL: <https://ui.shadcn.com/> (επίσκεψη 01/11/2024).
- [10] Alex Banks και Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*. 2nd. O'Reilly Media, 2020. ISBN: 978-1492051725.
- [11] pmndrs. *Zustand Documentation*. 2024. URL: <https://docs.pmnd.rs/zustand> (επίσκεψη 01/11/2024).
- [12] OpenAPI Initiative. *OpenAPI Specification*. 2024. URL: <https://spec.openapis.org/oas/latest.html> (επίσκεψη 01/11/2024).
- [13] Tom Brown κ.ά. “Language Models are Few-Shot Learners”. Στο: *Advances in Neural Information Processing Systems*. Τόμ. 33. 2020, σσ. 1877–1901.
- [14] Vercel. *Vercel AI SDK Documentation*. 2024. URL: <https://sdk.vercel.ai/docs> (επίσκεψη 01/11/2024).
- [15] OpenRouter. *OpenRouter Documentation*. 2024. URL: <https://openrouter.ai/docs> (επίσκεψη 01/11/2024).

- [16] Beds24. *Beds24 API Documentation v2.0*. 2024. URL: <https://beds24.com/api/v2/> (επίσκεψη 01/11/2024).
- [17] Rob Law, Daniel Leung και Ivan Chan. “Progress and Development of Information Technology in the Hospitality Industry: Evidence from Cornell Hospitality Quarterly”. Στο: *Cornell Hospitality Quarterly* 61.1 (2020), σσ. 19–45.
- [18] Erich Gamma κ.ά. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, 1994. ISBN: 978-0201633610.
- [19] Mark Richards και Neal Ford. *Fundamentals of Software Architecture: An Engineering Approach*. O’Reilly Media, 2020. ISBN: 978-1492043454.
- [20] Claudéric Demers. *dnd kit Documentation*. 2024. URL: <https://dndkit.com/> (επίσκεψη 01/11/2024).

Παράρτημα Α΄

Δομή Webhook Payload

Listing Α΄.1: Παράδειγμα Webhook Payload από Beds24

```
1 {  
2   "booking": {  
3     "id": 12345,  
4     "arrival": "2025-01-01",  
5     "departure": "2025-01-05",  
6     "status": "confirmed"  
7   },  
8   "messages": [...]  
9 }
```


Παράρτημα Β'

Παράδειγμα Auto Action Query

Listing Β'.1: Αναζήτηση κρατήσεων για before-checkin trigger

```
1  const whereClause = {  
2    'resource.checkInStartOfDay': {  
3      greater_than_equal: fromDate.toISOString(),  
4      less_than_equal: toDate.toISOString(),  
5    },  
6    'logging.automationTracking.triggeredAutomations': {  
7      not_in: [autoAction.id],  
8    }  
9  }
```